

Лабораторная работа 1

Классификация изображений с применением Transfer Learning

Цель работы: получить практические навыки по обучению распознавания образов, кластеризации и классификации данных с помощью обучения с учителем и метода transfer learning.

Необходимо:

- Любая среда для работы с Python
- Пакет Python Sklearn

Краткие теоретические сведения

Одним из наиболее распространенных способов наделяния машины искусственным интеллектом является машинное обучение. Методы машинного обучения в целом делятся на две категории: обучение с учителем и обучение без учителя.

Обучение с учителем (supervised learning; другое название: контролируемое обучение) — это процесс создания модели машинного обучения, основанной на маркированных (помеченных) обучающих (тренировочных) данных. Предположим, мы хотим создать систему, которая будет автоматически прогнозировать доход человека исходя из таких его характеристик, как возраст, образование, место жительства и т.п. Для этого мы должны создать базу данных, содержащую подробные сведения о людях, и пометить эти характеристики (признаки) соответствующими маркерами. Тем самым мы сообщаем нашему алгоритму, какие параметры соответствуют тому или иному доходу. На основании этого соответствия алгоритм будет учиться рассчитывать уровень дохода человека, используя предоставленные параметры.

Обучение без учителя (unsupervised learning; другие названия: неконтролируемое обучение, самообучение, спонтанное обучение) — это процесс создания модели машинного обучения, не основанный на использовании маркированных тренировочных данных. В некотором смысле это прямая противоположность процесса обучения с учителем, который обсуждался выше. В данном случае, ввиду отсутствия каких-либо отличительных меток, приходится обнаруживать отношения между данными, исходя лишь из самих данных. Предположим, мы хотим создать систему, предназначенную для разделения наборов точек данных на несколько групп. Проблема в том, что мы не можем точно указать, что именно должно использоваться в качестве критерия такого разделения. Следовательно, алгоритм обучения без учителя должен пытаться самостоятельно структурировать заданный набор данных путем разбиения на группы с использованием способа, который представляется наилучшим из всех возможных.

Что такое классификация

В этой главе мы обсудим методы классификации, основывающиеся на обучении с учителем. Сущность классификации заключается в разбиении данных на заданное количество классов. В ходе этого процесса мы распределяем данные по заданному фиксированному количеству категорий, чтобы обеспечить их наиболее целесообразное и эффективное использование.

В машинном обучении классификация решает задачу идентификации категории, к которой относится новая точка данных. Мы строим модель классификации на основании

тренировочного набора, содержащего точки данных и соответствующие метки. Предположим, мы хотим проверить, содержит ли данная фотография изображение лица человека. Мы создаем тренировочный набор, включающий данные двух классов: face и no-face. Затем мы тренируем модель, задействуя тренировочные образцы, которыми мы располагаем. После этого модель используется для выведения соответствующих умозаключений.

Хорошая классифицирующая система упрощает поиск и извлечение данных. Такие классификаторы широко применяются в системах распознавания лиц, идентификации спама, рекомендательных механизмах и т.п. Алгоритм классификации данных стремится определить наиболее подходящий критерий для разбиения данных на заданное количество классов.

Количество предоставляемых образцов должно быть достаточно большим, чтобы путем их обобщения можно было определить критерий разбиения данных на классы. При слишком малом количестве образцов алгоритм будет тяготеть в своих оценках к тренировочным данным. Это означает, что он не будет работать достаточно хорошо для неизвестных данных из-за переобучения модели на тренировочном наборе. В действительности в мире машинного обучения с этой проблемой приходится сталкиваться довольно часто. Этот фактор следует всегда учитывать при построении различных моделей машинного обучения.

1 Предварительная обработка данных

На практике мы имеем дело с большими объемами необработанных исходных данных. Алгоритмы машинного обучения рассчитаны на то, что, прежде чем они смогут начать процесс тренировки, получаемые данные будут отформатированы определенным образом. Чтобы привести данные к форме, приемлемой для алгоритмов машинного обучения, мы должны предварительно подготовить их и преобразовать в нужный формат. Покажем, как это делается.

Создайте новый файл Python и импортируйте следующие пакеты.

```
import numpy as np
from sklearn import preprocessing
```

Определим некоторую выборку данных.

```
input_data = np.array([[5.1, -2.9, 3.3],
                        [-1.2, 7.8, -6.1],
                        [3.9, 0.4, 2.1],
                        [7.3, -9.9, -4.5]])
```

Мы обсудим несколько различных методов предобработки данных:

- бинаризация
- исключение среднего
- масштабирование
- нормализация

Рассмотрим поочередно каждый из этих методов.

1.1 Бинаризация

Этот процесс применяется в тех случаях, когда мы хотим преобразовать наши числовые значения в булевы. Воспользуемся встроенным методом для бинаризации входных данных, установив значение 2.1 в качестве порогового.

Добавим следующие строки в тот же файл Python.

```
# Бинаризация данных
data_binarized = preprocessing.Binarizer(threshold=2.1).transform(input_data)
print("\nBinarized data:\n", data_binarized)
```

Выполнив этот код, вы получите следующий вывод, см. рис. 1. Можно видеть, что все значения свыше 2.1 принудительно устанавливаются равными единице, а остальные значения становятся равными нулю.

```
Binarized data:
[[ 1.  0.  1.]
 [ 0.  1.  0.]
 [ 1.  0.  0.]
 [ 1.  0.  0.]
```

Рисунок 1 — вывод бинаризованных данных

1.2 Исключение среднего

Исключение среднего — методика предварительной обработки данных, обычно используемая в машинном обучении. Как правило, из векторов признаков (feature vectors) целесообразно исключать средние значения, чтобы каждый признак (feature) центрировался на нуле. Это делается с той целью, чтобы исключить из рассмотрения смещение значений в векторах признаков.

Добавим следующие строки в тот же файл Python

```
# Вывод среднего значения и стандартного отклонения
```

```
print("\nBEFORE:")
print("Mean =", input_data.mean(axis=0))
print("Std deviation =", input_data.std(axis=0))
```

Две предыдущие строки отображают среднее значение и стандартное отклонение для входных данных. Исключим среднее.

```
# Исключение среднего
data_scaled = preprocessing.scale(input_data)
print("\nAFTER:")
print("Mean =", data_scaled.mean(axis=0))
print("Std deviation =", data_scaled.std(axis=0))
```

После выполнения этого кода в окне терминала отобразится следующая информация, см. рис. 2. Нетрудно заметить, что среднее значение практически равно нулю, а стандартное отклонение — единице.

```

BEFORE:
Mean = [ 3.775 -1.15 -1.3 ]
Std deviation = [ 3.12039661 6.36651396 4.0620192 ]
AFTER:
Mean = [ 1.11022302e-16 0.00000000e+00 2.77555756e-17]
Std deviation = [ 1. 1. 1.]

```

Рисунок 2 — вывод данных с исключенным средним

1.3 Масштабирование

В векторе признаков каждое значение может меняться в некоторых случайных пределах. Поэтому очень важно масштабировать признаки, чтобы они представляли собой равное игровое поле для тренировки алгоритма машинного обучения. Мы не хотим, чтобы любой из признаков мог принимать искусственно большое или малое значение лишь в силу природы измерений.

Добавим в тот же файл Python следующие строки.

```

# Масштабирование MinMax
data_scaler_minmax = preprocessing.MinMaxScaler(feature_range=(0, 1))
data_scaled_minmax = data_scaler_minmax.fit_transform(input_data)
print("\nMin max scaled data:\n", data_scaled_minmax)

```

После выполнения этого кода в окне терминала отобразится следующая информация, см. рис. 3. Каждая строка масштабирована так, чтобы максимальным значением было 1, а все остальные значения определялись относительно единицы.

```

Min max scaled data:
[[ 0.74117647  0.39548023  1.         ]
 [ 0.         1.         0.         ]
 [ 0.6        0.5819209   0.87234043]
 [ 1.         0.         0.17021277]]

```

Рисунок 3 — вывод масштабированных данных

1.4 Нормализация

Процесс нормализации заключается в изменении значений в векторе признаков таким образом, чтобы для их измерения можно было использовать одну общую шкалу. В машинном обучении используются различные формы нормализации. В наиболее распространенных из них значения изменяются так, чтобы их сумма была равна единице. L1-нормализация, использующая метод наименьших абсолютных отклонений (Least Absolute Deviations), обеспечивает равенство единице суммы абсолютных значений в каждом ряду. L2-нормализация, использующая метод наименьших квадратов, обеспечивает равенство единице суммы квадратов значений.

Вообще говоря, техника L1-нормализации считается более надежной по сравнению с L2-нормализацией, поскольку она менее чувствительна к выбросам. Очень часто данные содержат выбросы, и с этим ничего не поделаешь. Мы хотим использовать безопасные методики, позволяющие игнорировать выбросы в процессе вычислений. Если бы мы решали задачу, в которой выбросы играют важную роль, то, вероятно, лучшим выбором была бы L2-нормализация.

Добавим следующие строки в тот же самый файл Python.

Нормализация данных

```
data_normalized_l1 = preprocessing.normalize(input_data, norm='l1')
data_normalized_l2 = preprocessing.normalize(input_data, norm='l2')
print("\nL1 normalized data:\n", data_normalized_l1)
print("\nL2 normalized data:\n", data_normalized_l2)
```

Выполнив этот код, вы увидите в окне терминала следующий вывод, см. рис. 4.

```
L1 normalized data:
[[ 0.45132743 -0.25663717  0.2920354 ]
 [-0.0794702  0.51655629 -0.40397351]
 [ 0.609375   0.0625      0.328125 ]
 [ 0.33640553 -0.4562212  -0.20737327]]
L2 normalized data:
[[ 0.75765788 -0.43082507  0.49024922]
 [-0.12030718  0.78199664 -0.61156148]
 [ 0.87690281  0.08993875  0.47217844]
 [ 0.55734935 -0.75585734 -0.34357152]]
```

Рисунок 4 — вывод нормализованных данных

2 Кодирование меток

Как правило, в процессе классификации данных мы имеем дело со множеством меток (labels). Ими могут быть слова, числа или другие объекты. Функции машинного обучения, входящие в библиотеку `sklearn`, ожидают, что метки являются числами. Поэтому, если метки - это уже числа, мы можем использовать их непосредственно для того, чтобы начать тренировку. Однако обычно это не так.

На практике метками служат слова, поскольку в таком виде они лучше всего воспринимаются человеком. Мы помечаем тренировочные данные словами, чтобы облегчить отслеживание соответствий. Для преобразования слов в числа необходимо использовать кодирование. Под кодированием меток (label encoding) подразумевается процесс преобразования словесных меток в числовую форму. Благодаря этому алгоритмы могут оперировать нашими данными.

Создайте новый файл Python и импортируйте следующие пакеты.

```
import numpy as np
from sklearn import preprocessing
```

Определим метки.

```
# Предоставление меток входных данных
input_labels = ['red', 'black', 'red', 'green', 'black', 'yellow', 'white']
```

Создадим объект кодирования меток и обучим его.

```
# Создание кодировщика и установление соответствия между метками и числами
encoder = preprocessing.LabelEncoder()
encoder.fit(input_labels)
```

Выведем отображение слов на числа.

```
# Вывод отображения
print("\nLabel mapping:")
for i, item in enumerate(encoder.classes_):
    print(item, '-->', i)
```

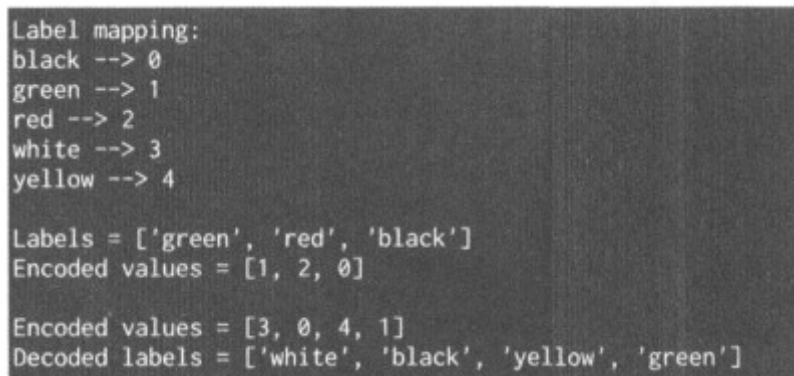
Преобразуем набор случайно упорядоченных меток, чтобы проверить работу кодировщика

```
# Преобразование меток с помощью кодировщика
test_labels = ['green', 'red', 'black']
encoded_values = encoder.transform(test_labels)
print("\nLabels =", test_labels)
print("Encoded values =", list(encoded_values))
encoder = preprocessing.LabelEncoder()
```

Декодируем случайный набор чисел.

```
# Декодирование набора чисел с помощью декодера
encoded_values = [3, 0, 4, 1]
decoded_list = encoder.inverse_transform(encoded_values)
print("\nEncoded values =", encoded_values)
print("Decoded labels =", list(decoded_list))
```

После выполнения этого кода в окне терминала должна отобразиться следующая информация, см. рис. 5. Легко увидеть, что все шаги кодирования и декодирования меток выполнены корректно.



```
Label mapping:
black --> 0
green --> 1
red --> 2
white --> 3
yellow --> 4

Labels = ['green', 'red', 'black']
Encoded values = [1, 2, 0]

Encoded values = [3, 0, 4, 1]
Decoded labels = ['white', 'black', 'yellow', 'green']
```

Рисунок 5 — вывод кодирования и декодирования меток

3 Логистический классификатор

Логистическая регрессия (logistic regression) — это методика, применяемая для объяснения отношений между входными и выходными переменными. Входные переменные считаются независимыми, выходные — зависимыми. Зависимая переменная может иметь лишь фиксированный набор значений, которые соответствуют классам задачи классификации.

Нашей целью является идентификация отношений между независимыми и зависимыми переменными посредством оценки вероятностей того, что та или иная зависимая переменная относится к тому или иному классу. Логистическая функция — это сигмоида, используемая для создания функций с различными параметрами. Она очень тесно связана с анализом данных на основе обобщенной линейной модели, в соответствии с которой делается попытка подогнать прямую линию к группе точек таким образом, чтобы минимизировать ошибку. Вместо линейной регрессии мы применяем логистическую регрессию. В действительности сама по себе логистическая регрессия предназначена не для классификации данных, однако она позволяет упростить решение этой задачи. Ввиду ее простоты логистическую регрессию часто задействуют в машинном обучении. Рассмотрим пример применения логистической регрессии для создания классификатора.

Создайте новый файл Python и импортируйте указанные ниже пакеты. Для удобства в пакете `utilities.py` содержится кастомная функция на основе `matplotlib.pyplot`, которую мы тоже импортируем.

```
import numpy as np
from sklearn import linear_model
import matplotlib.pyplot as plt

from utilities import visualize_classifier
```

Определим образец входных данных с помощью двумерных векторов и соответствующих меток.

```
# Определение образца входных данных
X = np.array([[3.1, 7.2], [4, 6.7], [2.9, 8], [5.1, 4.5], [6, 5], [5.6, 5], [3.3, 0.4], [3.9, 0.9], [2.8, 1],
[0.5, 3.4], [1, 4], [0.6, 4.9]])
y = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3])
```

Мы будем тренировать классификатор, используя эти помеченные данные. Создадим объект логистического классификатора.

```
# Создание логистического классификатора
classifier = linear_model.LogisticRegression(solver='liblinear', C=1)
```

Обучим классификатор, используя определенные выше данные.

```
# Тренировка классификатора
classifier.fit(X, y)
```

Визуализируем результаты работы классификатора, отследив границы классов

```
# Визуализация работы классификатора
visualize_classifier(classifier, X, y)
```

После выполнения этого кода на экране отобразится графическое окно, представленное на рис. 6.

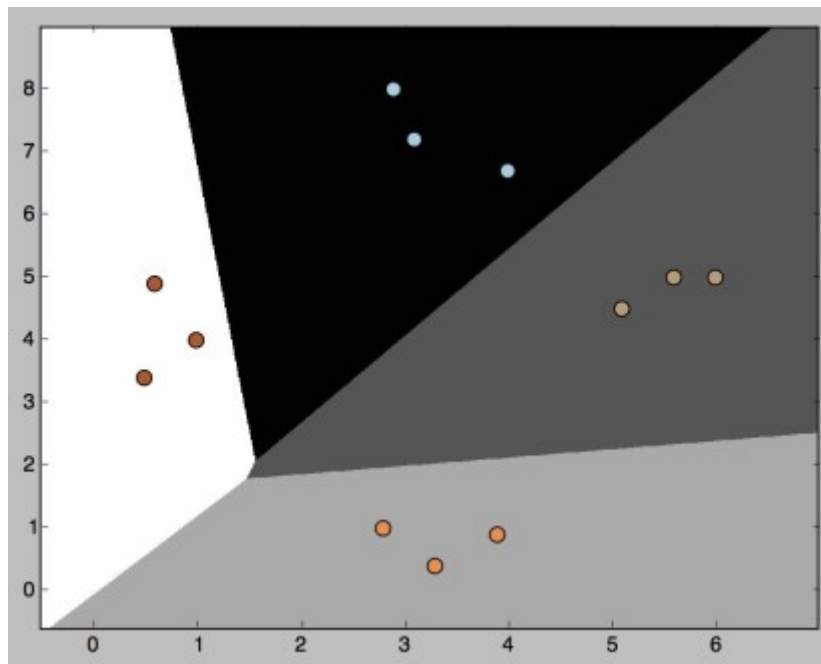


Рисунок 6 — вывод логистического классификатора.

Замените значение C на 100 и повторите выполнение кода. **Прикрепите к отчету выводы для $C=1$ и $C=100$. Проанализируйте результаты: что изменилось? Почему?**

4 Наивный байесовский классификатор

Наивный байесовский классификатор (Naive Bayes classifier) — это простой классификатор, основанный на использовании теоремы Байеса, которая описывает вероятность события с учетом связанных с ним условий. Такой классификатор создается посредством присваивания меток классов экземплярам задачи. Последние представляются в виде векторов значений признаков. При этом предполагается, что значение любого заданного признака не зависит от значений других признаков. Это предположение о независимости рассматриваемых признаков и составляет наивную часть байесовского классификатора.

Мы можем оценивать влияние любого признака переменной класса независимо от влияния других признаков. Например, мы можем считать животное гепардом, если оно имеет пятнистую кожу, четыре лапы и хвост и развивает скорость, равную примерно 70 миль в час. В случае использования наивного байесовского классификатора считается, что каждый из признаков вносит независимый вклад в конечный результат, оценивающий вероятность того, что данное животное является гепардом. Мы не будем утруждать себя рассмотрением корреляции между рисунком кожи, количеством лап, наличием хвоста и скоростью перемещения. Лучше займемся созданием наивного байесовского классификатора.

Создайте новый файл Python и импортируйте следующие пакеты.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn import cross_validation
```

```
from utilities import visualize_classifier
```

В качестве источника данных мы будем использовать файл data_multivar_nb.txt, каждая строка которого содержит значения, разделенные запятой.

```
# Входной файл, содержащий данные
input_file = 'data_multivar_nb.txt'
```

Загрузим данные из этого файла.

```
# Загрузка данных из входного файла
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

Создадим экземпляр наивного байесовского классификатора. В данном случае мы будем использовать гауссовский наивный байесовский классификатор, в котором предполагается, что значения, ассоциируемые с каждым классом, следуют закону распределения Гаусса.

```
# Создание наивного байесовского классификатора
classifier = GaussianNB()
```

Обучим классификатор, используя тренировочные данные.

```
# Тренировка классификатора
classifier.fit(X, y)
```

Запустим классификатор на тренировочных данных и спрогнозируем результаты.

```
# Прогнозирование значений для тренировочных данных
y_pred = classifier.predict(X)
```

Вычислим качество (accuracy) классификатора, сравнив предсказанные значения с истинными метками, а затем визуализируем результат.

```
# Вычисление качества классификатора
accuracy = 100.0 * (y == y_pred).sum() / X.shape[0]
print("Accuracy of Naive Bayes classifier =", round(accuracy, 2), "%")
```

```
# Визуализация результатов работы классификатора
visualize_classifier(classifier, X, y)
```

Предыдущий метод вычисления качества классификатора не является надежным. Нам нужно выполнить перекрестную проверку, чтобы не использовать те же самые тренировочные данные при тестировании.

Разобьем данные на обучающий и тестовый наборы. В соответствии со значением параметра test_size, указанным в строке кода ниже, мы отнесем 80% данных к тренировке, а оставшиеся 20% — к тестированию. Затем выполним тренировку наивного байесовского классификатора на этих данных.

```
# Разбивка данных на обучающий и тестовый наборы
X_train, X_test, y_train, y_test = cross_validation.train_test_split(X, y, test_size=0.2,
random_state=3)
classifier_new = GaussianNB()
classifier_new.fit(X_train, y_train)
y_test_pred = classifier_new.predict(X_test)
```

Вычислим качество классификатора и визуализируем результаты.

```
# Вычисление качества классификатора
accuracy = 100.0 * (y_test == y_test_pred).sum() / X_test.shape[0]
print("Accuracy of the new classifier =", round(accuracy, 2), "%")
```

```
# Визуализация работы классификатора
visualize_classifier(classifier_new, X_test, y_test)
```

Воспользуемся встроенными функциями для вычисления качества (accuracy), точности (precision) и полноты (recall) классификатора на основании тройной перекрестной проверки.

```
# Оценка показателей классификатора
num_folds = 3
accuracy_values = cross_validation.cross_val_score(classifier,
X, y, scoring='accuracy', cv=num_folds)
print("Accuracy: " + str(round(100*accuracy_values.mean(), 2)) + "%")
```

```
precision_values = cross_validation.cross_val_score(classifier,
X, y, scoring='precision_weighted', cv=num_folds)
print("Precision: " + str(round(100*precision_values.mean(), 2)) + "%")
```

```
recall_values = cross_validation.cross_val_score(classifier,
X, y, scoring='recall_weighted', cv=num_folds)
print("Recall: " + str(round(100*recall_values.mean(), 2)) + "%")
```

```
f1_values = cross_validation.cross_val_score(classifier,
X, y, scoring='f1_weighted', cv=num_folds)
print("F1: " + str(round(100*f1_values.mean(), 2)) + "%")
```

Выполнив этот код, вы получите изображение разбивки данных с помощью наивного байесовского классификатора и вывод информации о ключевых показателях классификатора. **Прикрепите их к отчету.**

5 Матрица неточностей

Матрица неточностей (confusion matrix) — это таблица, используемая для описания эффективности классификатора. Обычно она извлекается из тестового набора данных, для которого известны базовые истинные значения. При этом анализируются результаты отнесения каждого класса и определяется доля неверно отнесенных классов. В процессе конструирования матрицы неточностей мы фактически имеем дело с несколькими ключевыми метриками, играющими очень важную роль в машинном обучении. Обратимся к бинарной классификации, когда выходная переменная имеет два возможных значения: 0 и 1.

-
- **Истинноположительные результаты.** Это случаи, для которых мы предсказали 1 в качестве результата, и при этом истинный результат также равен 1.
- **Истинноотрицательные результаты.** Это случаи, для которых мы предсказали 0 в качестве результата, и при этом истинный результат также равен 0.
- **Ложноположительные результаты.** Это случаи, для которых мы предсказали 1 в качестве результата, тогда как истинный результат равен 0. Такая ситуация известна как ошибка 1 рода.
- **Ложноотрицательные результаты.** Это случаи, для которых мы предсказали 0 в качестве результата, тогда как истинный результат равен 1. Такая ситуация известна как ошибка 2 рода.

В зависимости от специфики конкретной задачи может потребоваться оптимизация алгоритма таким образом, чтобы уменьшить долю ложноположительных или ложноотрицательных результатов. Например, в системах биометрической идентификации личности очень важно избежать ложноположительных результатов, чтобы злоумышленники не могли получить доступ к конфиденциальной информации. Давайте посмотрим, как строится матрица неточностей.

Создайте новый файл Python и импортируйте следующие пакеты.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Определим выборки фактических (калибровочных) и предсказанных меток классов.

```
# Определение выборочных меток
true_labels = [2, 0, 0, 2, 4, 4, 1, 0, 3, 3, 3]
pred_labels = [2, 1, 0, 2, 4, 3, 1, 0, 1, 3, 3]
```

Построим матрицу неточностей, используя только что определенные метки.

```
# Построение матрицы неточностей
confusion_mat = confusion_matrix(true_labels, pred_labels)
```

Визуализируем матрицу неточностей.

```
# Визуализация матрицы неточностей
plt.imshow(confusion_mat, interpolation='nearest', cmap=plt.cm.gray)
plt.title('Confusion matrix')
plt.colorbar()
ticks = np.arange(5)
plt.xticks(ticks, ticks)
plt.yticks(ticks, ticks)
plt.ylabel('True labels')
plt.xlabel('Predicted labels')
plt.show()
```

В коде визуализации переменная `ticks` ссылается на количество различных классов. В нашем случае имеется пять различных меток.

Выведем отчет о результатах классификации.

```
# Отчет о результатах классификации
targets = ['Class-0', 'Class-1', 'Class-2', 'Class-3', 'Class-4']
print('\n', classification_report(true_labels, pred_labels, target_names=targets))
```

Выведенный отчет содержит оценки эффективности классификатора по отношению к каждому из классов. Выполнив код, вы получите изображение матрицы неточностей, а также терминальный вывод. **Прикрепите их к отчету и проанализируйте.**

6 Машины опорных векторов

Машина опорных векторов (Support Vector Machine, SVM) — это классификатор, определенный с использованием гиперплоскости, разделяющей классы. Разделяющая гиперплоскость представляет собой N-мерную версию прямой линии. При условии, что в задаче бинарной классификации задан маркированный обучающий набор данных, SVM находит оптимальную гиперплоскость, которая делит данные на два класса. Этот метод легко распространяется на задачу с N классами.

Рассмотрим двумерный случай с двумя классами точек. Таким образом, мы будем иметь дело только с точками и прямыми, расположенными на двумерной плоскости, что значительно проще, чем визуализировать векторы и гиперплоскости в пространстве большей размерности.

Рассмотрим рис. 7. Можно видеть два класса точек и пусть мы хотим найти оптимальную гиперплоскость, разделяющую эти классы. Однако что именно следует считать критерием оптимальности? На рисунке сплошная линия представляет оптимальную гиперплоскость. Можно провести множество прямых линий, разделяющих данные классы точек, но эта линия является наилучшим разделителем, поскольку она максимизирует расстояния, на которые точки удалены от разделяющей их линии. Точки, расположенные на пунктирных линиях, называются опорными векторами. Расстояние между двумя пунктирными линиями называется максимальным зазором.

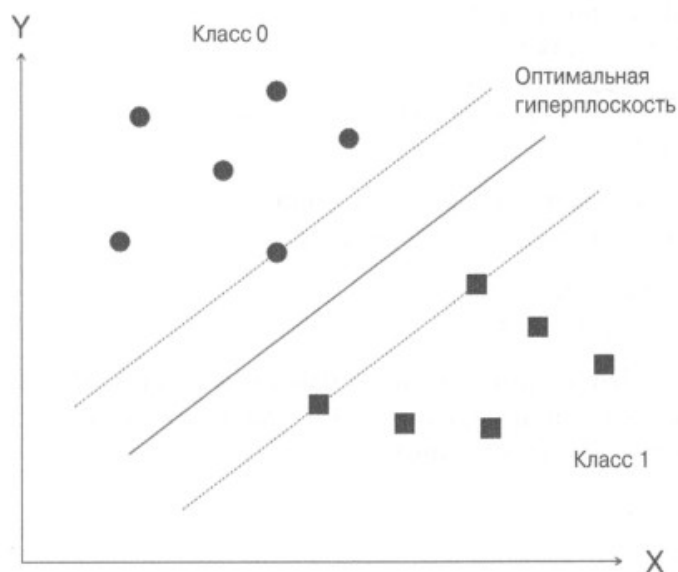


Рисунок 7 — пример решения задачи оптимальных векторов

Создадим классификатор в виде машины опорных векторов, предназначенный для прогнозирования границ дохода заданного физического лица на основе 14 атрибутов. Нашей целью является выяснение условий, при которых ежегодный доход человека превышает \$50 000 или меньше этой величины. Следовательно, мы имеем дело с задачей бинарной классификации. Набор данных возьмем из файла `income_data.txt`. Следует отметить одну особенность этого набора, которая заключается в том, что каждая точка данных представляет собой сочетание текста и чисел. Мы не можем использовать эти данные в необработанном виде, поскольку алгоритмам неизвестно, как обрабатывать слова. Мы также не можем преобразовать все данные, используя кодирование меток, поскольку числовые данные также содержат ценную информацию. Следовательно, чтобы создать эффективный классификатор, мы должны использовать комбинацию кодировщиков меток и необработанных числовых данных.

Создайте новый файл Python и импортируйте следующие пакеты и исследуемые данные.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import preprocessing
from sklearn.svm import LinearSVC
from sklearn.multiclass import OneVsOneClassifier
from sklearn import cross_validation
```

```
# Входной файл,
input_file = 'income_data.txt'
```

Загружаемые из файла данные нужно подготовить к классификации, подвергнув их предварительной обработке. Для каждого класса мы будем использовать не более 25 000 точек данных.

```
# Чтение данных
X = []
y = []
count_class1 = 0
count_class2 = 0
max_datapoints = 25000
```

Откроем файл и прочитаем строки.

```
with open(input_file, 'r') as f:
    for line in f.readlines():
        if count_class1 >= max_datapoints and count_class2 >= max_datapoints:
            break

        if '?' in line:
            continue

        data = line[:-1].split(' ')

        if data[-1] == '<=50K' and count_class1 < max_datapoints:
            X.append(data)
            count_class1 += 1
```

```

if data[-1] == '>50K' and count_class2 < max_datapoints:
    X.append(data)
    count_class2 += 1

```

Каждая строка данных отделяется от следующей с помощью запятой, что требует соответствующего разбиения строк. Последним элементом каждой строки является метка. В зависимости от этой метки мы будем относить данные к тому или иному классу.

Преобразуем список в массив numpy array, чтобы его можно было использовать в качестве входных данных функций sklearn.

```

# Преобразование в массив numpy array
X = np.array(X)

```

Если атрибут — строка, то он нуждается в кодировании. Если атрибут — число, мы можем оставить его в том виде, как он есть. Заметьте, что в конечном счете мы получим несколько кодировщиков меток, которые нам нужно будет отслеживать.

```

# Преобразование строковых данных в числовые
label_encoder = []
X_encoded = np.empty(X.shape)
for i,item in enumerate(X[0]):
    if item.isdigit():
        X_encoded[:, i] = X[:, i]
    else:
        label_encoder.append(preprocessing.LabelEncoder())
        X_encoded[:, i] = label_encoder[-1].fit_transform(X[:, i])

```

```

X = X_encoded[:, :-1].astype(int)
y = X_encoded[:, -1].astype(int)

```

Создадим SVM-классификатор с линейным ядром.

```

# Создание SVM-классификатора
classifier = OneVsOneClassifier(LinearSVC(random_state=0))

```

Обучим классификатор.

```

# Обучение классификатора
classifier.fit(X, y)

```

Выполним перекрестную проверку, разбив данные на обучающий и тестовый наборы в пропорции 80/20, а затем спрогнозируем результат для тренировочных данных.

```

# Перекрестная проверка
X_train, X_test, y_train, y_test = cross_validation.train_test_split(X, y, test_size=0.2,
random_state=5)
classifier = OneVsOneClassifier(LinearSVC(random_state=0))
classifier.fit(X_train, y_train)
y_test_pred = classifier.predict(X_test)

```

Вычислим для классификатора F-меру.

```
# Вычисление F-меры для SVM-классификатора
f1 = cross_validation.cross_val_score(classifier, X, y, scoring='f1_weighted', cv=3)
print("F1 score: " + str(round(100*f1.mean(), 2)) + "%")
```

Теперь, имея подготовленный классификатор, посмотрим, что произойдет, если мы выберем некоторую случайную точку данных и предскажем для нее результат. Определим одну такую точку.

```
# Предсказание результата для тестовой точки данных
input_data = ['37', 'Private', '215646', 'HS-grad', '9', 'Never-married', 'Handlers-cleaners', 'Not-in-family', 'White', 'Male', '0', '0', '40', 'United-States']
```

Прежде чем приступить к прогнозированию, мы должны присвоить этой точке данных код, используя созданный ранее кодировщик признаков.

```
# Кодирование тестовой точки данных
input_data_encoded = [-1] * len(input_data)
count = 0
for i, item in enumerate(input_data):
    if item.isdigit():
        input_data_encoded[i] = int(input_data[i])
    else:
        input_data_encoded[i] = int(label_encoder[count].transform(input_data[i]))
        count += 1
```

```
input_data_encoded = np.array(input_data_encoded)
```

Теперь мы готовы к тому, чтобы спрогнозировать результат с помощью классификатора.

```
# Выполнение классификации для кодированной точки данных и вывод результата
predicted_class = classifier.predict(input_data_encoded)
print(label_encoder[-1].inverse_transform(predicted_class)[0])
```

Тренировка классификатора посредством этого кода займет несколько секунд. После выполнения кода в окне терминала отобразится F-мера:

F1 score: 66.82%

Кроме того, для тестовой точки данных отобразится следующий вывод: <=50к

Проверив значения в этой точке данных, можно убедиться в том, что она близко соответствует точкам данных, принадлежащим классу с доходом менее \$50 000.

Поиграйте с характеристиками работы классификатора (F-мера, точность, полнота), используя различные ядра и тестируя несколько комбинаций параметров. **Прикрепите к отчету несколько результатов для разных точек данных (не менее трех).**

7 Регрессия

Регрессия — это процесс оценки того, как соотносятся между собой входные и выходные переменные. Следует отметить, что выходные переменные могут иметь значения из непрерывного ряда вещественных чисел. Следовательно, существует бесконечное количество

результатирующих возможностей. Это резко контрастирует с процессом классификации, в котором количество выходных классов фиксировано.

В регрессии предполагается, что выходные переменные зависят от входных, и наша задача заключается в выяснении соотношения между ними. Отсюда входные переменные называют независимыми переменными (или предикторами), а выходные — зависимыми (или критериальными переменными). При этом вовсе не требуется, чтобы входные переменные были независимы друг от друга. Существует множество ситуаций, когда между входными переменными существует корреляция.

Регрессионный анализ позволяет выяснить, как изменяется значение выходной переменной, когда мы изменяем лишь часть входных переменных, оставляя остальные входные переменные фиксированными. В случае линейной регрессии предполагается, что входные и выходные переменные связаны между собой линейной зависимостью. Это налагает ограничения на нашу процедуру моделирования, но ускоряет ее и делает более эффективной.

Иногда линейной регрессии оказывается недостаточно для объяснения соотношений между входными и выходными переменными. В подобных случаях используют полиномиальную регрессию, в которой входные и выходные переменные связаны между собой полиномиальной зависимостью. С вычислительной точки зрения такой подход более сложен, но обеспечивает более высокую точность. Выбор вида регрессии для выявления указанных соотношений определяется конкретикой задачи. Регрессию часто используют для прогнозирования цен, экономических показателей и т.п.

7.1 Создание регрессора одной переменной

Приступим к построению регрессионной модели на основе одной переменной. Создайте новый файл Python и импортируйте следующие пакеты.

```
import pickle
```

```
import numpy as np  
from sklearn import linear_model  
import sklearn.metrics as sm  
import matplotlib.pyplot as plt
```

Мы также будем использовать файл `data_singlevar_regr.txt`, содержащий исходные данные.

```
# Входной файл, содержащий данные  
input_file = 'data_singlevar_regr.txt'
```

В этом текстовом файле в качестве разделителя используется запятая, поэтому для загрузки данных можно воспользоваться следующим вызовом функции.

```
# Загрузка данных  
data = np.loadtxt(input_file, delimiter=',')  
X, y = data[:, :-1], data[:, -1]
```

Разобьем данные на обучающий и тестовый наборы.

```
# Разбивка данных на обучающий и тестовый наборы
```

```
num_training = int(0.8 * len(X))
num_test = len(X) — num_training
```

```
# Тренировочные данные
X_train, y_train = X[:num_training], y[:num_training]
```

```
# Тестовые данные
X_test, y_test = X[num_training:], y[num_training:]
```

Создадим объект линейного регрессора и обучим его, используя тренировочные данные.

```
# Создание объекта линейного регрессора
regressor = linear_model.LinearRegression()
```

```
# Обучение модели с использованием обучающего набора
regressor = linear_model.LinearRegression()
```

Спрогнозируем результат для тестовой набора данных, используя обучаемую модель.

```
# Прогнозирование результата
y_test_pred = regressor.predict(X_test)
```

Построим выходной график.

```
# Построение графика
plt.scatter(X_test, y_test, color='green')
plt.plot(X_test, y_test_pred, color='black', linewidth=4)
plt.xticks(())
plt.yticks(())
plt.show()
```

Вычислим метрические характеристики регрессора, сравнивая истинные значения с предсказанными.

```
# Вычисление метрик
print("Linear regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred), 2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pred), 2))
print("Median absolute error =", round(sm.median_absolute_error(y_test, y_test_pred), 2))
print("Explain variance score =", round(sm.explained_variance_score(y_test, y_test_pred), 2))
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))
```

Создав модель, мы можем сохранить ее в файле для последующего использования. Python предоставляет отличный модуль pickle, который позволяет легко это сделать.

```
# Файл для сохранения модели
output_model_file = 'model.pkl'
```

```
# Сохранение модели
with open(output_model_file, 'wb') as f:
    pickle.dump(regressor, f)
```

Загрузим модель из файла на диске и построим прогноз.

```
# Загрузка модели
with open(output_model_file, 'rb') as f:
    regressor_model = pickle.load(f)

# Получение прогноза на тестовом наборе данных
y_test_pred_new = regressor_model.predict(X_test)
print("\nNew mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred_new), 2))
```

Выполнив этот код, вы получите изображение результатов предсказания и метрики в терминале. **Прикрепите их к отчету и проанализируйте результаты.**

7.2 Создание многомерного регрессора

Рассмотрим теперь работу с многомерными данными. Создайте новый файл Python и импортируйте следующие пакеты.

```
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
from sklearn.preprocessing import PolynomialFeatures
```

Мы также будем использовать файл data_multivar_regr.txt.

```
# Входной файл, содержащий данные
input_file = 'data_multivar_regr.txt'
```

В этом текстовом файле в качестве разделителя используется запятая, поэтому для загрузки данных можно воспользоваться следующим вызовом функции.

```
# Загрузка данных
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

Разобьем данные на обучающий и тестовый наборы.

```
# Разбивка данных на обучающий и тестовый наборы
num_training = int(0.8 * len(X))
num_test = len(X) — num_training
```

```
# Тренировочные данные
X_train, y_train = X[:num_training], y[:num_training]
```

```
# Тестовые данные
X_test, y_test = X[num_training:], y[num_training:]
```

Создадим и обучим модель линейного регрессора.

```
# Создание модели линейного регрессора
linear_regressor = linear_model.LinearRegression()
```

```
# Обучение модели с использованием обучающих наборов
linear_regressor.fit(X_train, y_train)
```

Спрогнозируем результат для тестового набора данных.

```
# Прогнозирование результата
y_test_pred = linear_regressor.predict(X_test)
```

Выведем метрические характеристики.

```
# Измерение метрик
```

```
print("Linear Regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred), 2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pred), 2))
print("Median absolute error =", round(sm.median_absolute_error(y_test, y_test_pred), 2))
print("Explained variance score =", round(sm.explained_variance_score(y_test, y_test_pred), 2))
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))
```

Создадим полиномиальный регрессор степени 10 и обучим его на тренировочных данных. Возьмем некоторую выборочную точку данных и спрогнозируем для нее результат. Первый шаг заключается в том, чтобы преобразовать ее в полином.

```
# Полиномиальная регрессия
polynomial = PolynomialFeatures(degree=10)
X_train_transformed = polynomial.fit_transform(X_train)
datapoint = [[7.75, 6.35, 5.56]]
poly_datapoint = polynomial.fit_transform(datapoint)
```

Нетрудно заметить, что эта точка весьма близка к точке данных [7.66, 6.29, 5.66], указанной в строке 11 нашего файла данных. Поэтому удачно созданный регрессор должен предсказать результат, близкий к 41.35. **Создайте объект линейного регрессора и выполните подгонку к полиному. Постройте прогноз с использованием как линейного, так и полиномиального регрессоров, чтобы увидеть разницу.**

```
poly_linear_model = linear_model.LinearRegression()
poly_linear_model.fit(X_train_transformed, y_train)
print("\nLinear regression:\n", linear_regressor.predict(datapoint))
print("\nPolynomial regression:\n", poly_linear_model.predict(poly_datapoint))
```

После выполнения этого кода в окне терминала отобразится информация о метриках. **Прикрепите их к отчету и проанализируйте.**

8 Оценка стоимости недвижимости с использованием регрессора на основе машины опорных векторов

В пакете `sklearn` имеются встроенные наборы данных, одним из которых мы воспользуемся здесь для создания регрессора, позволяющего оценивать стоимость жилья. Каждая точка набора данных определяется 13 атрибутами. Наша задача заключается в прогнозировании

стоимости жилья на основе значений этих атрибутов. Создайте новый файл Python и импортируйте следующие пакеты.

```
import numpy as np
from sklearn import datasets
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, explained_variance_score
from sklearn.utils import shuffle

data = datasets.load_boston()
```

Перемешаем данные, чтобы усилить объективность анализа

```
# Перемешивание данных
X, y = shuffle(data.data, data.target, random_state=7)
```

Разобьем данные на обучающий и тестовый наборы в соотношении 80/20.

```
# Разбивка данных на обучающий и тестовый наборы
num_training = int(0.8 * len(X))
X_train, y_train = X[:num_training], y[:num_training]
X_test, y_test = X[num_training:], y[num_training:]
```

Создадим и обучим SVM-регрессор, используя линейное ядро. Параметр *C* представляет собой штраф за ошибки обучения. Если вы увеличите значение *C*, модель более точно подстроится под обучающие данные. Однако это может привести к переобучению модели и потере ее общности. Параметр *epsilon* задает пороговое значение: штраф за ошибки в обучении не налагается, если расхождение между фактическим и предсказанным значениями не превышает этой величины.

```
# Создание регрессионной модели на основе SVM
sv_regressor = SVR(kernel='linear', C=1.0, epsilon=0.1)
```

```
# Обучение регрессора SVM
sv_regressor.fit(X_train, y_train)
```

Оценим эффективность работы регрессора и вычислим метрики.

```
# Оценка эффективности работы регрессора
y_test_pred = sv_regressor.predict(X_test)
mse = mean_squared_error(y_test, y_test_pred)
evs = explained_variance_score(y_test, y_test_pred)
print("\n#### Performance ####")
print("Mean squared error =", round(mse, 2))
print("Explained variance score =", round(evs, 2))
```

Выберем тестовую точку данных и спрогнозируем для нее результат.

```
# Тестирование регрессора на тестовой точке данных
test_data = [3.7, 0, 18.4, 1, 0.87, 5.95, 91, 2.5052, 26, 666, 20.2, 351.34, 15.27]
print("\nPredicted price:", sv_regressor.predict([test_data])[0])
```

После выполнения этого кода в окне терминала отобразится вывод, прикрепите его к отчету. **Поиграйте с параметрами C и ϵ , прикрепите к отчету несколько результатов (не менее трех) для разных параметров и разных точек данных. Проанализируйте результаты.**

Полезные ссылки.

Подборка баз данных для обучения распознавания изображений

<https://habr.com/ru/companies/magnus-tech/articles/954130/#%D0%9A%D0%BB%D0%B0%D1%81%D1%81%D0%B8%D1%84%D0%B8%D0%BA%D0%B0%D1%86%D0%B8%D1%8F%20%D0%B8%D0%B7%D0%BE%D0%B1%D1%80%D0%B0%D0%B6%D0%B5%D0%BD%D0%B8%D0%B9>

Цель работы: сформировать навыки использования предобученных нейросетевых архитектур для решения задач компьютерного зрения на специфических наборах данных.

Задачи:

1. Выбрать датасет для задачи классификации изображений (не менее 3-х классов), согласовать выбор с преподавателем.
2. Выбрать предобученную архитектуру (ResNet, EfficientNet или MobileNet) и обосновать выбор.
3. Реализовать стратегию «Feature Extraction»: заморозить веса базовой сети, заменить выходной слой и обучить модель. Оценить точность и построить матрицу ошибок (confusion matrix).
4. Реализовать стратегию «Fine-tuning»: разморозить часть верхних слоев базовой сети и провести дообучение с низким темпом обучения (learning rate).
5. Сравнить результаты двух подходов (метрики Accuracy, Precision, Recall) и сделать вывод о целесообразности дообучения весов базовой сети.

Формат сдачи работы: отчет в формате PDF, оформленный по актуальному ГОСТу по оформлению ВКР в Университете ИТМО. К отчету прикладывается ссылка на репозиторий с исходным кодом (Github или Google Collab).

Критерии оценивания:

- Практическая работа отражает выполнение всех поставленных задач.
- Обучающийся способен ответить на вопросы по архитектуре выбранной сети и механизму Transfer Learning с приведением примеров.
- Сдача практической работы в установленный срок.
- Отчет оформлен в соответствии с требованиями.

Основаниями для снижения количества баллов за одну задачу в диапазоне от 0,5 до 2 является:

- Не полностью выполнены задачи (например, отсутствует сравнение стратегий).

- Ответы на вопросы не получены и/или ответы на вопросы частично полные без примеров и пояснений.
- Работа сдана после срока.
- Отчет не соответствует требованиям.

Содержание отчета

Требуется подготовить отчеты в формате DOC\DOCX или PDF. Отчет содержит титульный лист, постановку задач и артефакты их выполнения.

Артефакты лабораторной работы:

1. Ссылка на репозиторий со всеми программами, обсуждаемыми в пп. 1-8.
2. Изображения выводов логистического классификатора из п.3 с параметрами $C=1, 100$, а так же анализ этих результатов.
3. Изображение разбивки данных с помощью наивного байесовского классификатора и вывод информации о ключевых показателях классификатора из п.4, а так же анализ этих результатов.
4. Изображение матрицы неточностей, и терминальный вывод из п.5, а так же анализ этих результатов.
5. Терминальные выводы предсказания уровня дохода людей с помощью классификатора на опорных векторах из п.6. Не менее трех различных точек данных.
6. Изображение результатов предсказания, терминальный вывод метрик для одномерного регрессора из п. 7.1, а так же анализ этих результатов.
7. Прогноз с использованием линейного и полиномиального многомерных регрессоров из п. 7.2, а так же анализ этих результатов.
8. Прогноз результатов оценки стоимости жилья для различных точек данных (не менее трех) с использованием регрессора на опорных векторах из п. 8 с различными параметрами C и $epsilon$, а также анализ полученных результатов.

Отчет необходимо выслать в течение 4-х недель на адрес d.tzarev@itmo.ru.

В теме письма: No группы ФИО (латинскими буквами) No работы (например: 5555 Fedor Sumkin 1)